

ADVANCED SOFTWARE DEVELOPMENT

SOFTWARE ESTIMATION

Abdulfattah Alfarra

Outline

- ▣ Definition
- ▣ Cost Estimating Methodologies
 - Price to Win
 - Expert Opinion,
 - Analogy,
 - Algorithmic cost modelling
- ▣ Agile Cost Estimation
 - Determining the Effort
 - Determining Agile Velocity
 - Completion Time
 - Cost

Estimation

- ▣ The National Estimating Society has defined Cost Estimating as:
 - The art of approximating the probable cost of something based on information available at the time
- ▣ Cost estimating is a powerful tool because it:
 - Leads to a better understanding of the problem,
 - Improves management insight into resource allocation problems, and
 - Provides an objective baseline to measure progress.

Software cost estimation

- ▣ Software cost estimation is the process of predicting the effort required to develop a software system.
- ▣ In recent years, software has become the most expensive component of computer system projects.

Software cost estimation

- ▣ Accurate cost estimation is important because:
 - It can help to classify and prioritize development projects with respect to an overall business plan.
 - It can be used to determine what resources to commit to the project and how well these resources will be used.
 - It can be used to assess the impact of changes and support replanning.
 - Projects can be easier to manage and control when resources are better matched to real needs.
 - Customers expect actual development costs to be in line with estimated costs.

Fundamental estimation questions

- ▣ How much effort is required to complete an activity?
- ▣ How much calendar time is needed to complete an activity?
- ▣ What is the total cost of an activity?
- ▣ Project estimation and scheduling are interleaved management activities.

Cost Estimating Methodologies

- ▣ Once data has been collected and normalized to constant dollars, there are five methodologies available for estimating costs:
 - Price to Win
 - Expert Opinion,
 - Analogy,
 - Algorithmic cost modelling

Pricing to win

أقل الاسعار لكسب العقد

- ▣ The project costs whatever the customer has to spend on it.
- ▣ Advantages:
 - You get the contract.
- ▣ Disadvantages:
 - The probability that the customer gets the system he or she wants is small. Costs do not accurately reflect the work required.

Pricing to Win

- ▣ This approach may seem unethical and un-business like.
- ▣ However, when detailed information is lacking it may be the only appropriate strategy.
- ▣ The project cost is agreed on the basis of an outline proposal and the development is constrained by that cost.
- ▣ A detailed specification may be negotiated or an evolutionary approach used for system development.

Expert Opinion

- ▣ Often called Delphi method, proposed by Dr. Barry Boehm in his book, Software Engineering Economics.
- ▣ Useful in assessing differences between past projects and new ones for which no historical precedent exists.

Expert Opinion

▣ Advantages:

- Little or no historical data needed.
- Suitable for new or unique projects.

▣ Disadvantages:

- Very subjective.
- Experts may introduce bias.
 - ▣ Larger number of experts will help to reduce bias
- Qualification of experts may be questioned.

Expert Opinion - Steps

- Gather a group of experts together,
- Describe overall program in enough detail so experts can provide an estimate,
- Each member of the expert group then does an independent of the resources needed,
- Estimates are gathered anonymously and compared,
- If there exists significant divergence among the estimates, the estimates will be returned to the expert group,

Expert Opinion - Steps

- ▣ The expert group then discusses the estimates and the divergence and works to resolve differences, and
- ▣ The expert group once again submits anonymous, independent estimates which continues until a stable estimate results.

Analogy

مقارنة مع مشاريع سابقة
مقارنة مع نصف مشروع او اكثر من مشروع

- ▣ Estimates costs by comparing proposed programs with similar, previously completed programs for which historical data is available.
 - Actual costs of similar existing system are adjusted for complexity, technical, or physical differences to derive new cost estimates
 - Analogies are used early in a program cycle when there is insufficient actual cost data to use as a detailed approach
 - Compares similarities and differences

Analogy

قد احتاج لخبير

- ▣ Often use single historical data point.
- ▣ May have several analogy estimates of sub elements to make up the total cost.
- ▣ Comparison process is key to success.
 - May have to add or delete functionality from historical costs to match new program
- ▣ Consult technical experts for advice on complexity factors, technical, performance or physical differences.
 - (not to be confused with expert opinion method)
- ▣ Impact of differences on cost is subjective.

Analogy

- ▣ Good choice for:
 - A new system that is derived from an existing subsystem.
 - ▣ Make sure actual cost data is available
 - A system where technology/programmatic assumptions have advanced beyond any existing cost estimating relationships.
- ▣ Provides link between technical assumptions and cost.

Analogy

- ▣ Advantages:
 - Inexpensive
 - Easily changed
 - Based on actual experience (of the analogous system)
- ▣ Disadvantages:
 - Very Subjective
 - Large amount of uncertainty
 - Truly similar projects must exist and can be hard to find
 - Must have detailed technical knowledge of program and analogous system

Algorithmic cost modelling

- ▣ Cost is estimated as a mathematical function of product, project and process attributes whose values are estimated by project managers.
- ▣ Equations are based on theory or historical data
- ▣ Use input such as SLOC, number of functions to perform and other cost drivers
- ▣ تقدير عدد الوظائف عدد الاوبجكتس عدد الشاشات

Algorithmic cost modelling

▣ Examples:

■ COCOMO (COConstructive COSt MOdel)

- ▣ Developed by Boehm in 1981
- ▣ Became one of the most popular and most transparent cost model
- ▣ Mathematical model based on the data from 63 historical software project
- ▣ مجموعة من المعادلات الرياضية لحساب المعطيات

■ COCOMO II

- ▣ Published in 1995
- ▣ To address issue on non-sequential and rapid development process models, reengineering, reuse driven approaches, object oriented approach etc
- ▣ Has three submodels – application composition, early design and post-architecture
- ▣ لا يستخدموا الا في الوتر فول ميثودولوجي

Algorithmic cost modelling

▣ Advantages

- Generate repeatable estimations
- Easy to modify input data
- Easy to refine and customize formulas
- Objectively calibrated to experience
- أدق الأنواع

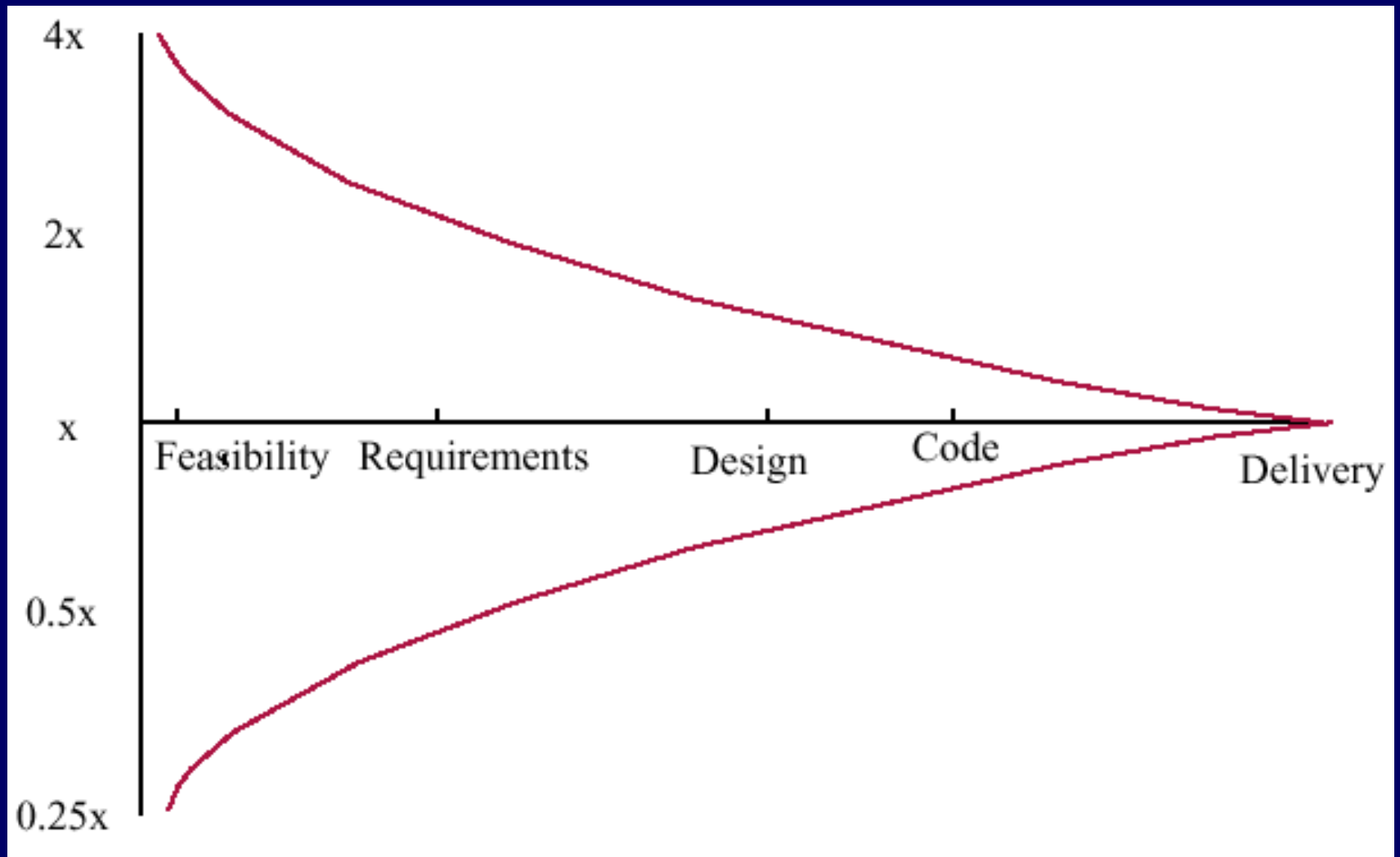
▣ Disadvantages

- Unable to deal with exceptional conditions
- Some experience and factors can not be quantified

Estimation accuracy

- ▣ The size of a software system can only be known accurately when it is finished
- ▣ Several factors influence the final size
 - Use of COTS and components
 - Programming language
 - Distribution of system
- ▣ As the development process progresses then the size estimate becomes more accurate

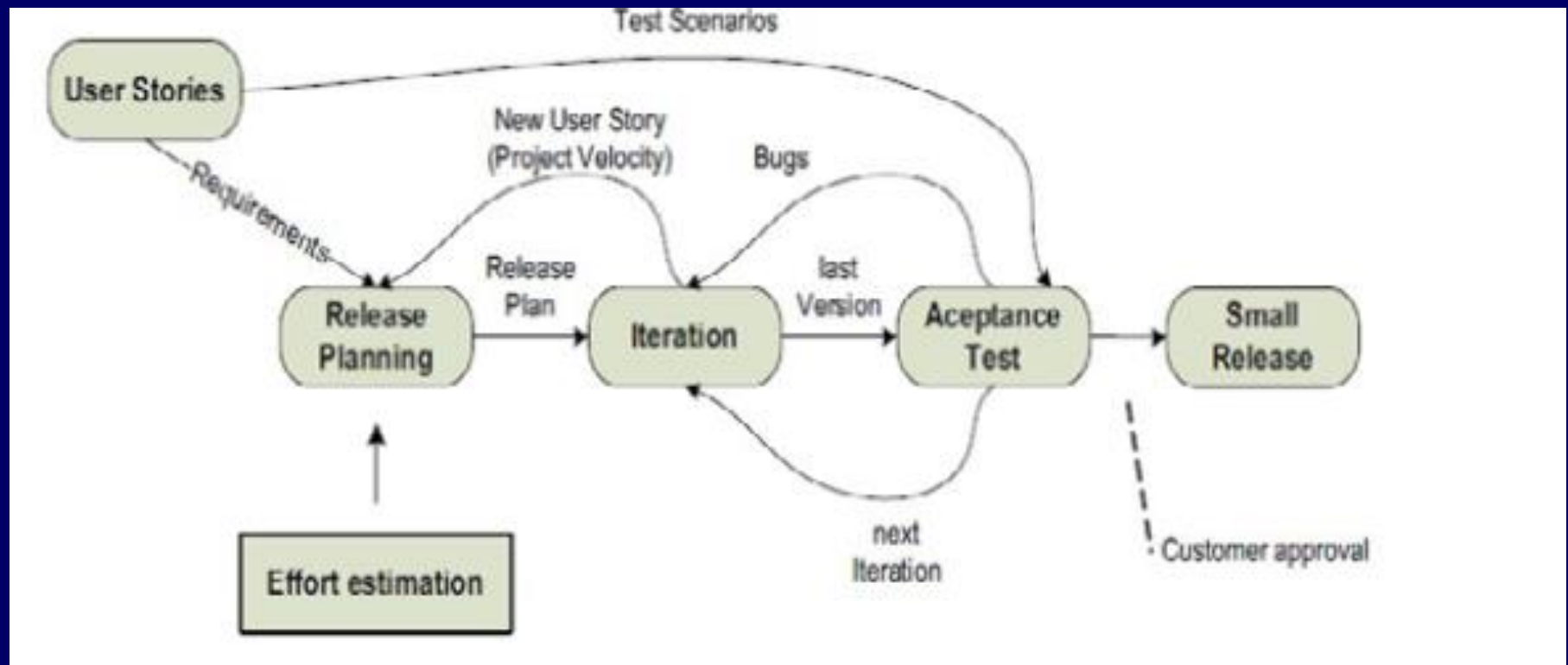
Estimate uncertainty



Agile Cost Estimation

- ▣ Effort estimation model is needed to predict development effort of Agile software project, based on characteristics of Agile Software Development methodology.

Agile Cost Estimation



Agile Cost Estimation

- ▣ In waterfall a team member's workload capacity is determined by the manager who estimates how long certain tasks will take and then assigns work based on that team member's total available time
- ▣ Agile methodology takes a considerably different approach to determining a team member's capacity. First of all, it assigns work to an entire team, not an individual.

Agile Cost Estimation

- ▣ Secondly, team members use effort and degree of difficulty to estimate their own work.
- ▣ Therefore, no need to estimate Personnel requirements for the project, thus the model is intended to calculate Completion Time and Cost for the Agile Software project.

Agile Cost Estimation

Determining the Effort

- ▣ We keep the vectors to two so as to keep the process as simple as possible.

Story Size

- ▣ Story size is an estimate of the relative scale of the work in terms of actual development effort. Table 1 shows five values, assigned to different types of user stories according to their size.

Determining the Effort

Value	Guidelines
5	• An extremely large story • Too large to accurately estimate • Should almost certainly be broken down into a set of smaller Stories • May be a candidate for separation into a new project
4	• Requires the focused effort of a developer for a long period of time – Think in terms of more than a week of work • Should consider breaking it down into a set of smaller stories

Determining the Effort

Value	Guidelines
3	• A moderately large story • Think in terms of two to five days of work
2	• Think in terms of a roughly a day or two of work
1	• A very small story representing tiny effort level. • Think in terms of only a few hours of work.

Determining the Effort

- ▣ **Complexity**
- ▣ This is complexity of either or both the requirements of the Story and or its technical complexity. Complexity introduces uncertainty to the estimate – more complexity means more uncertainty. Table 2 shows 5 values, assigned to user stories according to their nature.

Determining the Effort

Value	Guidelines
5	• Extremely complex • Many dependencies on other stories, other systems or subsystems • Represents a skill set or experience that is important, but absent in the team • Story is difficult to accurately describe • Many unknowns • Requires significant refactoring • Requires extensive research • Requires difficult judgment calls • Effects of the Story have significant impact external to the story itself

Determining the Effort

Value	Guidelines
4	• Very complex • Multiple dependencies on other stories, other systems or subsystems • Represents a skill set or experience that is important, but not strong in the team • Story is somewhat difficult for product owner to accurately describe • Multiple unknowns • Comparatively large amount of refactoring required • Requires research • Requires senior level programming skills to complete • Requires somewhat difficult judgment calls • Effects of the Story have moderate impact

Determining the Effort

Value	Guidelines
3	• Moderately complex • Moderate number of dependencies on other stories, other systems or subsystems • Represents a skill set or experience that is reasonably strong in the team • Story is somewhat difficult for owner to accurately describe • Moderate level of unknowns • Some refactoring may be required • Requires intermediate programming skills to complete • Requires little research • Requires few important judgment calls • Effects of the Story have minimal impact external to the story itself

Determining the Effort

Value	Guidelines
2	• Easily understood technical and business requirements • Little or no research required • Few unknowns • Little if any research required • Requires basic to intermediate programming skills to complete • Effects of the Story are almost completely localized to the Story itself

Determining the Effort

Value	Guidelines
1	• Very straightforward with few if any unknowns • Technical and business requirements very clear with no ambiguity • No unknowns • No research required • Requires basic programming skills to complete • Effects of Story are completely localized to the Story itself

Determining the Effort

- ▣ Using these two vectors, **effort** of a particular User Story is determined using the following simple formula:
- ▣ $ES = \text{Complexity} \times \text{Size}$
- ▣ Effort for the complete project will be sum of efforts of all individual user stories.

$$E = \sum_{i=1}^n (ES)_i$$

Determining the Effort

Example

- ▣ If we have 13 user stories
- ▣ 9 moderate large story and moderate complexity
- ▣ 2 small size Moderately complex
- ▣ 2 Extremely large size and Very straightforward

$$\begin{aligned} \text{ES} &= (9 * 3 * 3) + (2 * 1 * 3) + (2 * 5 * 1) \\ &= 97 \end{aligned}$$

Effort = 97

Determining Agile Velocity

- ▣ The Observed Velocity is simply how many Units of Effort your team completes in a typical Sprint.
- ▣ To measure the project velocity you simply add up the estimates of the user stories that were finished during the iteration. It's just that simple. You also total up the estimates for the tasks finished during the iteration.

Determining Agile Velocity

- ▣ The calculation of Velocity in physics is pretty straightforward, i.e.
- ▣ $\text{Velocity} = \text{Distance} / \text{Time}$
- ▣ For our purposes, the distance is Units of Effort and Time (the denominator) is the length of our Sprint. Velocity is thus calculated:
- ▣ $V_i = \text{Units of Effort Completed} / \text{Sprint Time}.$

Determining Agile Velocity

- ▣ If team velocity (number of days for all user stories) is 40
- ▣ And sprint (fixed iteration time) is 10
- ▣ Then $v_i = 40/10 = 4$

Optimization of Velocity

- ▣ Optimization is a process that should be completed before you begin Calibration.
- ▣ There are two things of primary interest to us in our calibration. They are:
 - i. The **Friction** or consistent forces that are a constant drag on productivity and reduce Project Velocity.
 - ii. The Variable or **Dynamic Forces** that decelerate the project or team members and cause the Project Velocity to be irregular.

Optimization of Velocity

i) *Friction:*

- ▣ In Software development, there are countless forces that can affect the Velocity of your team.
- ▣ Optimizing them before Calibration is important. Friction forces include:

Optimization of Velocity

- ▣ **Team composition:** Are the right people with the right skills on the team.
- ▣ **Process:** Changes to your processes: Agile methods, build, release, testing, etc...
- **Environmental factors:** Interruptions, noise, poor ventilation, poor lighting, uncomfortable seating and desks, inadequate hardware or software, etc...
- **Team dynamics:** Some team members may not play nicely with others.

Friction Factors

Friction Factor	Stable	Volatile	Highly Volatile	Very Highly Volatile
Team Composition	1	0.98	0.95	0.91
Process	1	0.98	0.94	0.89
Environmental Factors	1	0.99	0.98	0.91
Team Dynamics	1	0.98	0.91	0.85

Optimization of Velocity

- ▣ Friction (FR) is calculated as product of all four fraction factors (FF).

$$FR = \prod_{i=1}^4 (FF)_i$$

Optimization of Velocity

- ▣ Example
- ▣ Friction Factors
- ▣ Team Composition 0.95
- ▣ Process 0.89
- ▣ Environmental Factors 0.98
- ▣ Team Dynamics 0.85
- ▣ $FF = 0.95 * 0.89 * 0.98 * 0.85$
- ▣ $FF = 0.70$

Optimization of Velocity

Variable or Dynamic Forces

- ▣ Variable or Dynamic forces are often unpredictable and unexpected. They decelerate your project and cause a loss of Velocity. Their effects are sometimes dramatic, but their influence is often brief.
- ▣ The more consistent and predictable the force, the more consistent your Velocity will be.

Optimization of Velocity

- ▣ **Team changes:** Adding member, losing members, changing roles and responsibilities.
- ▣ **New tools:** Introduction of new development tools, database technologies, languages, etc... require learning, and reduce Velocity until learned.
- ▣ **Vendor defects:** Defects in third party tools and software requiring developer workarounds eat into productivity and Velocity.
- ▣ **Responsibilities outside of the project:** Team members assuming additional responsibilities outside of the project. Shifting between projects can have a dramatic effect on productivity.

Optimization of Velocity

- ▣ **Personal issues:** Colicky baby at home, personal health, family dynamics, etc...
- ▣ **Stakeholders:** Stakeholders may not be responsive to requests for information from the developers or tester and thus create delays. They may also have unreasonable expectations of the team.
- ▣ **Unclear requirements:** Lack of clarity or detail in requirements cause unnecessary churn and rework.
- ▣ **Changing requirements:** New project specifications might require skills that are non-existent or weak in the team. Acquiring the skills, either by introducing new team members, or by an existing team member acquiring the skills will impact productivity.
- ▣ • **Relocation:** Moving the team to a new physical location upsets the rhythm and impacts their Velocity

Dynamic Force Factors

Variable Factor	Normal	High	Very High	Extra High
Expected Team Changes	1	0.98	0.95	0.91
Introduction of New Tools	1	0.99	0.97	0.96
Team member's responsibilities outside the project	1	0.98	0.94	0.90
Personal Issues	1	0.99	0.98	0.98

Dynamic Force Factors

Variable Factor	Normal	High	Very High	Extra High
Expected Delay in Stakeholder response	1	0.99	0.98	0.95
Expected Ambiguity in Details	1	0.98	0.97	0.95
Expected Changes in environment	1	0.99	0.98	0.97
Expected Relocation	1	0.99	0.99	0.98

Dynamic Force Factors

Dynamic Force (DF) is calculated as product of all nine variable factors (VF) ▣

$$DF = \prod_{i=1}^9 (VF)_i$$

Dynamic Force Factors

Example:

- ▣ **Dynamic Force Factors Expected Team Changes**
0.98
- ▣ Introduction of New Tools 0.97
- ▣ Vendor's Defect 0.94
- ▣ Team member's responsibilities 0.98
- ▣ Personal Issues 0.98
- ▣ Expected Delay in Stakeholder response 0.96
- ▣ Expected Ambiguity in Details 0.98
- ▣ Expected Changes in environment 0.97
- ▣ Expected Relocation 0.98
- ▣ $DF = 0.97 * 0.94 * 0.98 * 0.98 * 0.96 * 0.98 * 0.97 * 0.98$
- ▣ $DF = 0.77$

Deceleration

- ▣ **Deceleration** is the rate of negative change of velocity. In our case, deceleration is the product of Friction and Dynamic Forces affecting the velocity. It is calculated as

$$D = FR \times DF$$

- ▣ Example
- ▣ $D = 0.70 \times 0.77$
- ▣ $D = 0.54$

Completion Time

Completion Time

▣ In order to estimate duration needed to complete the project, it is calculated as:

$$T = \frac{E}{V} \text{ Days}$$
$$= \frac{\sum_{i=1}^n (ES)_i}{(V_i)^D} \text{ Days}$$

$$T = 97 / (4)^{0.54}$$

$$T = 97 / 2.1 = 46.02 \text{ Days}$$

Completion Time

- ▣ The unit of T in this calculation is Days which can be then converted to Months, dividing by No. of Working Days per month, Thus

$$T = \frac{\sum_{i=1}^n (ES)_i}{(V_i)^D} * \frac{1}{WD} \text{ Months}$$

- ▣ Given : No of Working days per Month = 22
- ▣ $T = 46.02 / 22 = 2.1$ month.

Cost

- ▣ $COST = 1.681 * TS * T$
- ▣ 1.681 is factor for other expenses than salary such as equipments, rents, furniture,...etc.
- ▣ Where TS is monthly Team Salary and T is calculated Time in Months.

Cost

Example:

- ▣ Monthly Team Salary = 500000
- ▣ $\text{Cost} = 1.681 * 50000 * 2.1$
- ▣ $\text{Cost} = 176505$